

- Q1. (a) (i) $O(n^{0.5})$
 (ii) $O(2^{1.5n})$
 (iii) $O(n!)$
 (iv) $O(n^2)$
 (v) $O(\log n)$
 (vi) $O(n \log \log n)$

(b) $\exists c=2000, n_0=1 \bullet \forall n > n_0 \bullet$
 $n \geq 1$
 $n^{0.01} \geq 1$
 $n^{4.01} \geq n^4$
 $n^4 \leq n^{4.01}$
 $2000n^4 \leq 2000n^{4.01}$
 $2000n^4 \leq cn^{4.01}$

- (c) Need to prove $[\exists c, n_0 \bullet \forall n > n_0 \bullet 2000n^4 \geq cn^{4.01}]$ is wrong.
 Equivalent to prove for $[\forall c, n_0 \bullet \exists n > n_0 \bullet 2000n^4 < cn^{4.01}]$
 For any c, n_0 , take $n = \max(n_0, \lceil (2000/c)^{100} \rceil) + 1$, we have

$$n > (2000/c)^{100}$$

$$n^{0.01} > 2000/c$$

$$2000 < cn^{0.01}$$

$$2000n^4 < cn^{4.01}$$

- (d) $\forall c \bullet (\exists) \text{ take } n_0 = \max(\lceil (16^{16})/(15! c) \rceil, 16) \bullet \forall n > n_0$
 we have $(16^{16})/(15! c) < n$ replacing n_0 by $(16^{16})/(15! c)$
 $\Rightarrow (16^{16}) < c(15!)n$ rewriting
 $\Rightarrow (16^{16}) (16^{n-16}) < c(15!)n(16^{n-16})$ multiplying 16^{n-16} to both sides
 $\Rightarrow 16^n < c(15!)n(16)(17)\dots(n-1)$ replacing 16^{n-16} by $(16)(17)\dots(n-1)$
 $\Rightarrow 16^n < c(1)(2)\dots(15)(16)(17)\dots(n-1)(n)$ rewriting
 $\Rightarrow 2^{4n} < c(n!)$ rewriting

Q2. (a) $T(n) = 2T(n/2) + n$

- (i) By substitution.

Claim: $T(n) \leq cn \log n$, for some $c \geq 1, n > 2$

$$T(n)$$

$$= 2T(n/2) + n$$

$$\leq 2c(n/2)\log(n/2) + n$$

$$= cn(\log n - 1) + n$$

$$= cn \log n - cn + n$$

$$\leq cn \log n$$

- (ii) By master method

$$a=2, b=2, \log_b a = 1, f(n) = n.$$

$$\exists k=0 \bullet$$

$$f(n) = n = [n^{\log_2 2}] \log^0 n = [n^{\log_b a}] \log^k n$$

$$f(n) = \Theta(n^{\log_b a} \log^k n).$$

$$\text{Case 2 applies. } T(n) = \Theta(n^{\log_b a} \log^{k+1} n) = \Theta(n \log n).$$

(iii) In Mergesort, we divide the set into two “equal” parts, and this explains 2 and $n/2$. The merge algorithm need $O(n)$ time.

(iv) The student is wrong.

The correct one is $T(n) = \max_{k=1 \text{ to } n} \{T(k-1) + T(n-k)\} + n$

Claim: $T(n) < c n^2$, $c \geq 1$, $n \geq 2$.

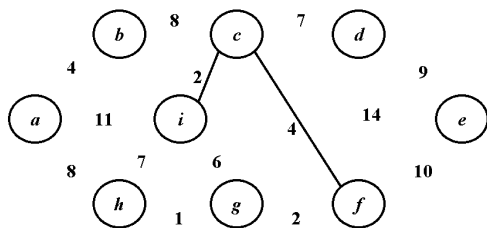
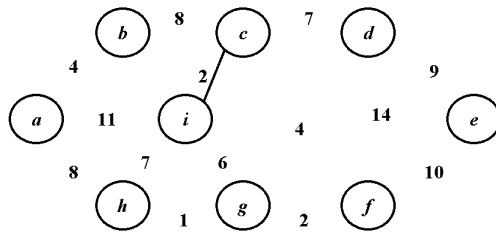
Choose a value for k , say h , such that $T(k-1) + T(n-k)$ is maximum. Then, $T(n) = T(h-1) + T(n-h) + n < c(h-1)^2 + c(n-h)^2 + n < c h^2 + c(n-h)^2 + n < c h^2 + c(n-h)^2 + 2c(h)(n-h) = c n^2$.

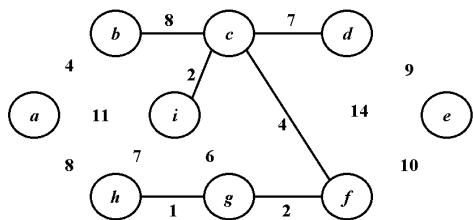
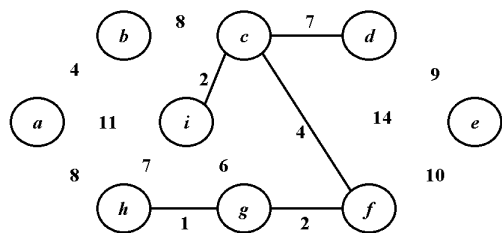
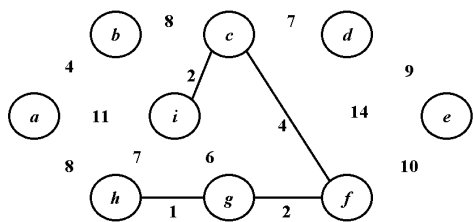
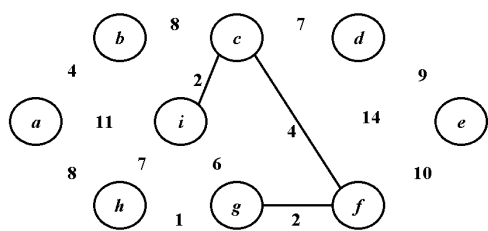
$2c(h)(n-h) - n > 0$ because $2c(h)(n-h)$ is minimum when $h=1$ or $n-1$.

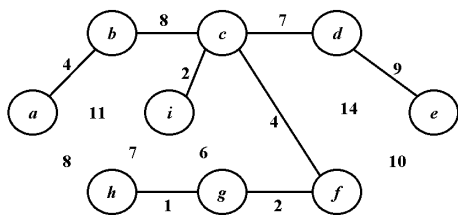
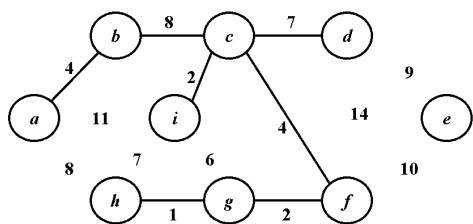
We take $c \geq 1$, $n \geq 2$.

(b) In the execution of Euclid’s algorithm, $\text{gcd}(m,n)$ calls $\text{gcd}(n, m \bmod n)$, $\text{gcd}(n, m \bmod n)$ calls $\text{gcd}(m \bmod n, n \bmod (m \bmod n))$, ..., etc. The first parameter drops more than half for every two recursive calls. Hence, time complexity = $O(\log m)$.

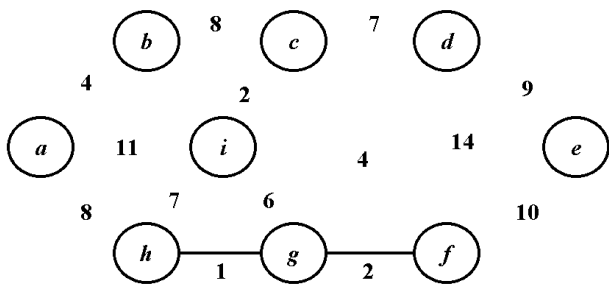
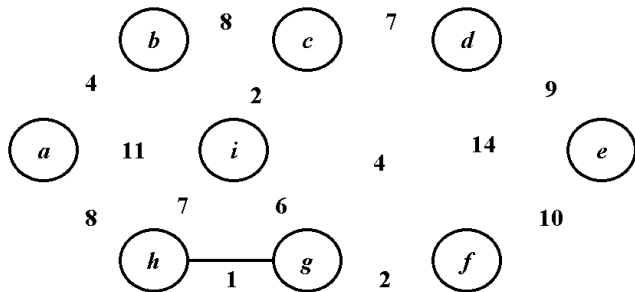
Q3. (a)

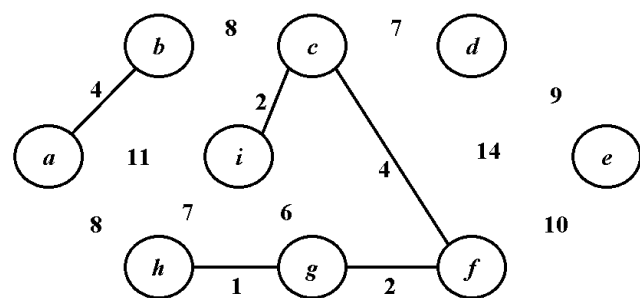
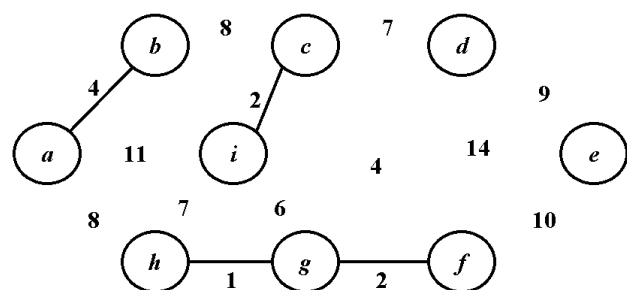
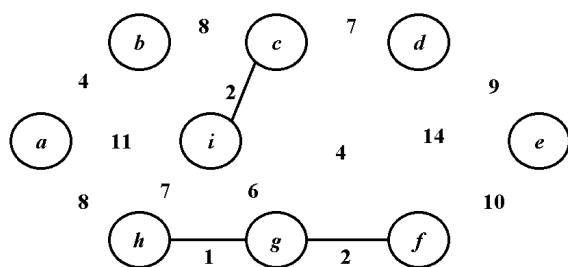


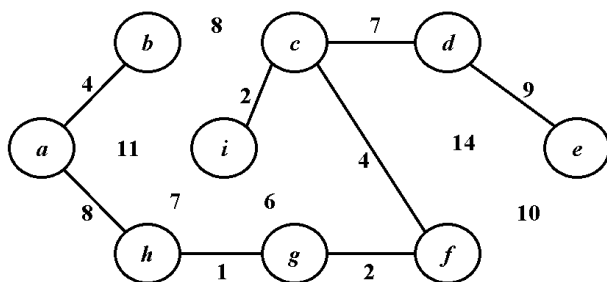
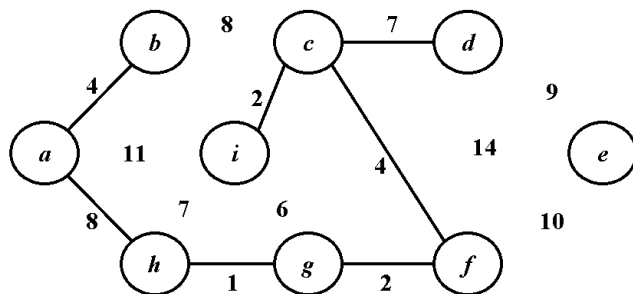
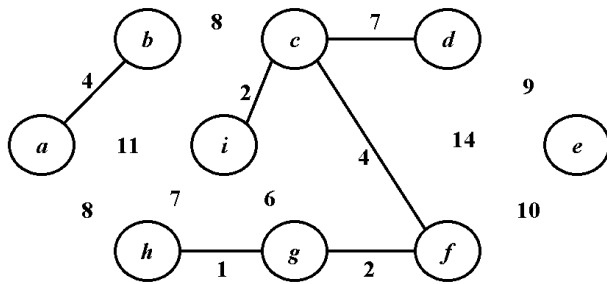




(b)







- c) Change all weights to their logarithms, and apply Prim's or Kruskal algorithm to the new graph.

Q4. (a) Define profit(r,d).

$\forall r, d$, where $0 \leq r \leq R$ and $0 \leq d \leq D$,
if $r=0$ or $d=0$, profit(r,d)=0; Else, profit(r,d) =
 $\max_{i=0 \text{ to } r} (300i+500(r-i)+\text{profit}(r-\lceil 0.3i \rceil - \lceil 0.6(r-i) \rceil, d-1)$;

i = no of robots assigned to A
 $300i+500(r-i)=500r-200i$ is the profit earned.

(b)

```
Algorithm profit(r,d)
if r=0 or d=0
    return 0;
p=0;
for i=0 to r
    temp=(300i+500(r-i)+profit(r-⌈0.3i⌉-⌈0.6(r-i)⌉, d-1);
    if p<temp
        p=temp;
return p;
```

(c)

```
Construct a R x D table, T[0..R, 0..D].
for d=0 to D
    T[0,d]=0;
for r=0 to R
    T[r,0]=0;

for d=1 to D
    for r=1 to R
        p=0;
        for i=0 to r
            temp=(300i+500(r-i)+T(r-⌈0.3i⌉-⌈0.6(r-i)⌉, d-1);
            if p<temp
                p=temp;
        T[r,d]=p;
```

Q5. (a) Base case: N=1.

$p=r$ is true in the first if-condition. Bubble() returns without doing anything.
As there is only one element, the sorted property is implied.
Therefore, the statement is true for the base case.

Assume the statement is true for $N=k \geq 1$.

Consider the statement for $N=k+1$.

After the execution of the for-loop, the maximum element (or the last maximum element if it is not unique) will move to r .

By inductive assumption, Bubble(A,p,r-1) will sort the array A[p...r-1] in ascending order. Together with the maximum element at A[r], A[p...r] is sorted ascendingly.

Result follows from induction.

- (b) Let N be $r-p+1$.
- (i) $N-1$. First, there exists a scenario that $N-1$ elements moving towards r . Second, there cannot be N elements moving towards r . At least one will move towards p .
 - (ii) $N/2$. If one element x moves towards r , at least one element y will move towards p , which swaps with x . The element y cannot swap with any other element again.